ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

Курсовая работа

по дисциплине

«Объектно-Ориентированное Программирование»

по теме

«Создание графического интерфейса для заданий 1-4 на языке программирования Java»

Выполнил
Студент гр. в5130904/30322 Александров Н. В.
Преподаватель Маслаков А. П.

Санкт-Петербург
2025 г.

Содержание

Введение 2

Постановка задачи 3

Структура проекта 4

Результаты выполнения программы 6

- Лабораторная работа 1 6
- Лабораторная работа 2 7
- Лабораторная работа 3 8
- Лабораторная работа 4 10

Выводы 11

Код проекта 13

Постановка задачи

Разработать приложение с графическим интерфейсом для заданий 1–4.

Для этого приложения должна быть реализована возможность выбора из списка любого приложения, ввод входных данных и его выполнение.

Модифицировать задания 1–4 так, чтобы весь вывод происходил в текстовых

областях, защищённых от редактирования. Предусмотреть для заданий:

• 3 - выбор файлов словаря и текста для перевода, возможность ручного ввода текста;

• 4 - ввод входных данных для методов.

Блок практических заданий 1.1-1.4 призван сформировать у студента

понимание особенностей хранения, умение настраивать и поддерживать данные.

Структура проекта

На рисунке 1 изображена структура проекта:

1. Папка java содержит четыре подпапки: lab_1, lab_2, lab_3, и lab_4. Каждая из этих папок включает в себя код, реализующий задания для соответствующих лабораторных работ.
2. Папка coursework содержит основной класс программы Main и MainWindowController, реализующий проект курсовой работы.
3. Папка resources содержит внешние файлы, использующиеся для работы программы.

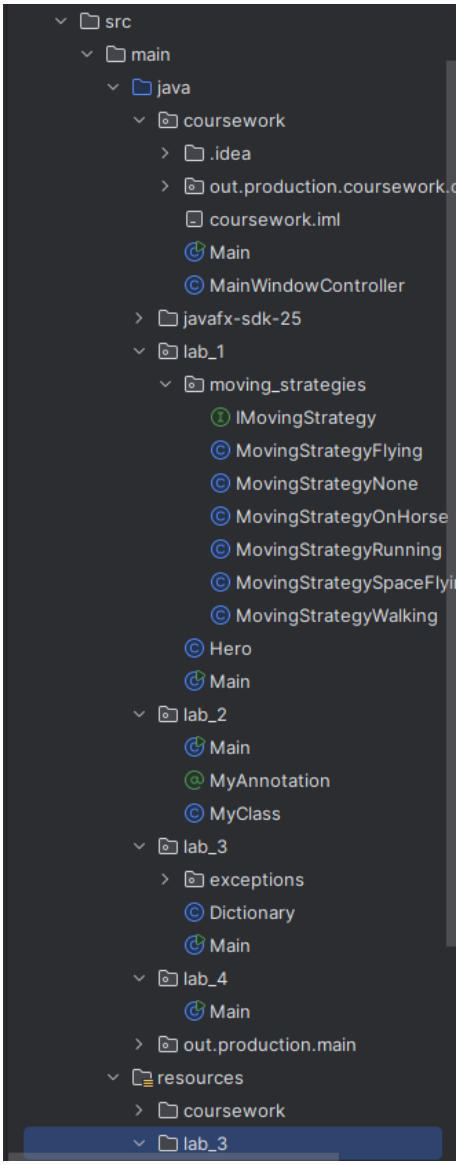


Рисунок 1. Структура проекта

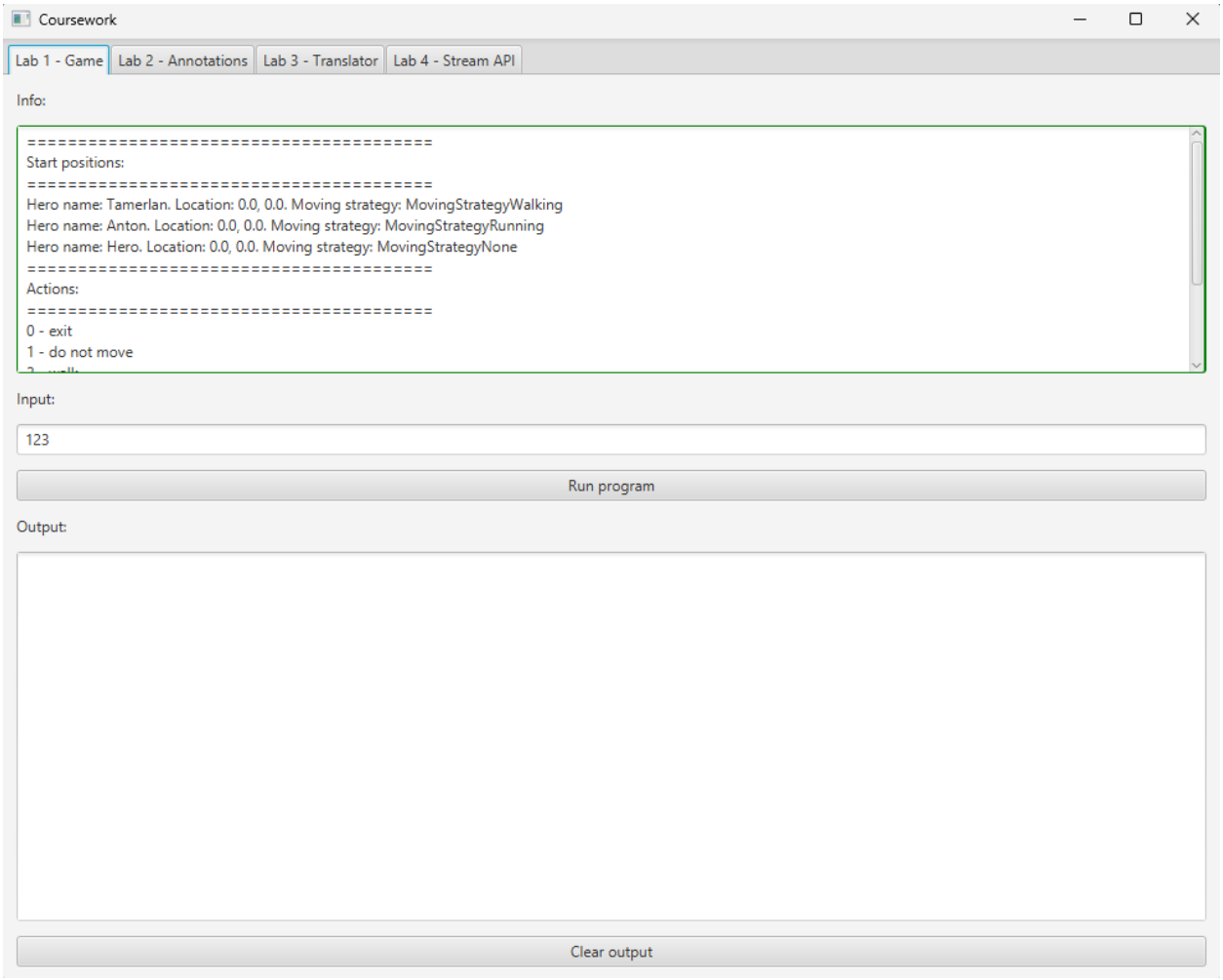


Рисунок 2. Интерфейс программы

Результаты выполнения

Лабораторная работа 1 - Game

Допустим введем в поле выбора действий последовательную команду 350, где 3 – бежать, 5 – космический полет, 0 – прекращение игры. Посмотрим на результат на рисунке 3 ниже. При этом если ввести некорректную команду в поле выбора действия, то программа, выдаст ошибку, которая показана на рисунке 4.

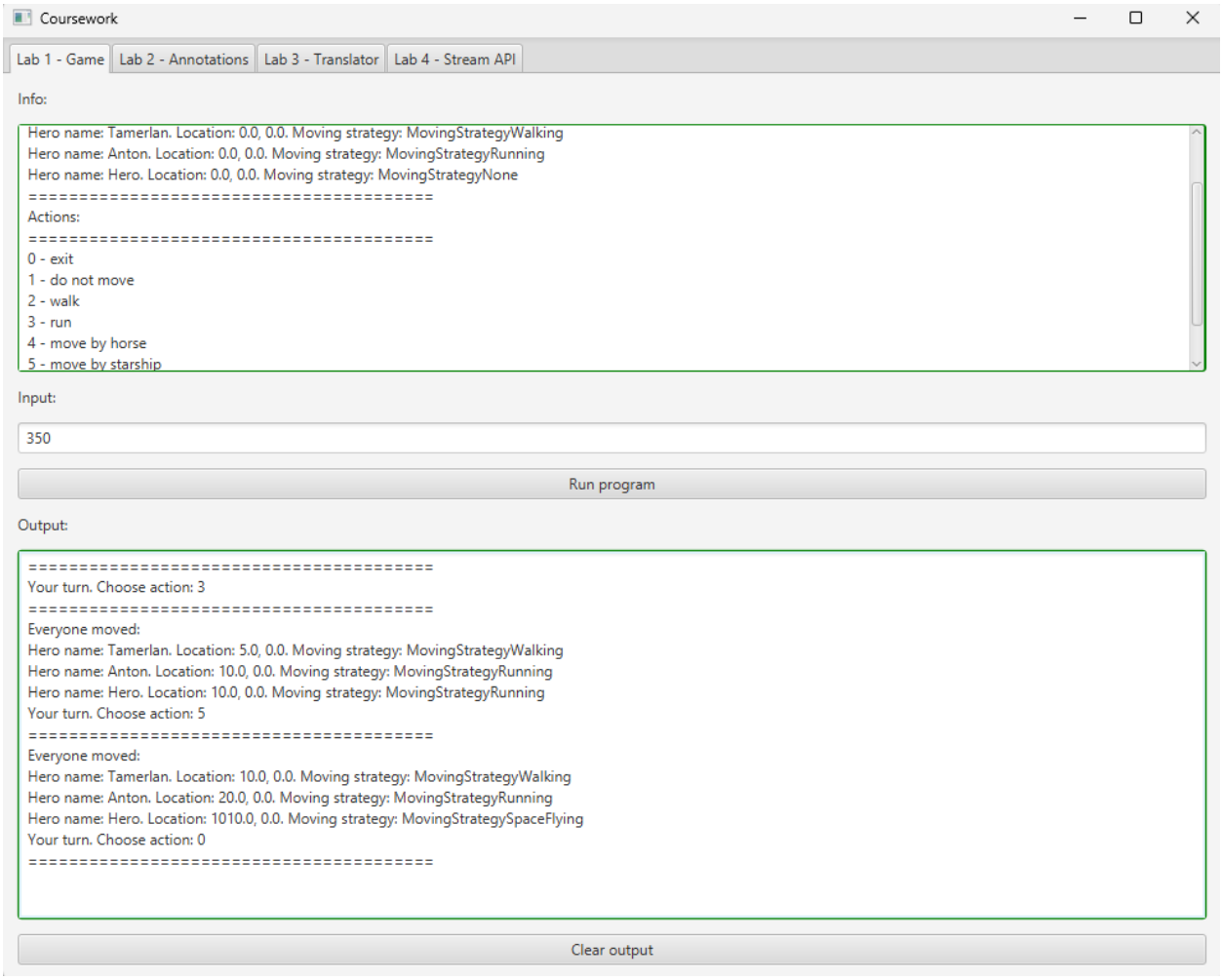


Рисунок 3. Результат выполнения лабораторной работы №1.

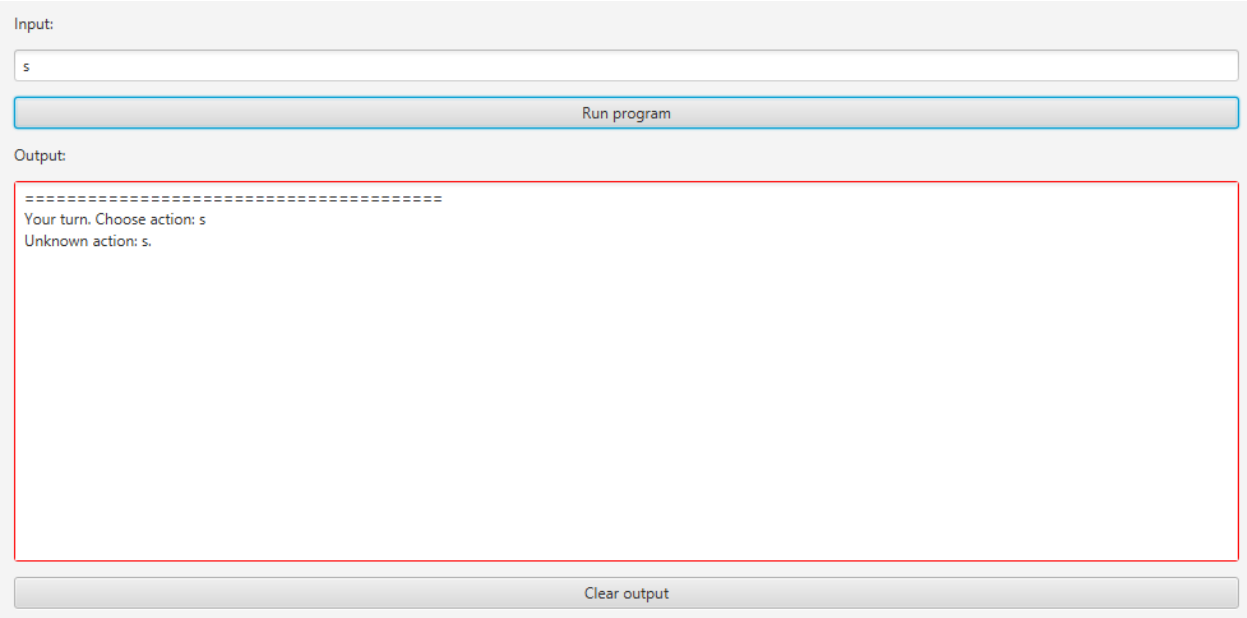


Рисунок 4. Обработка некорректного ввода в лабораторной работе №1.

Лабораторная работа 2 - Annotations

Результат выполнения лабораторной работы №2 показан на рисунке 5.

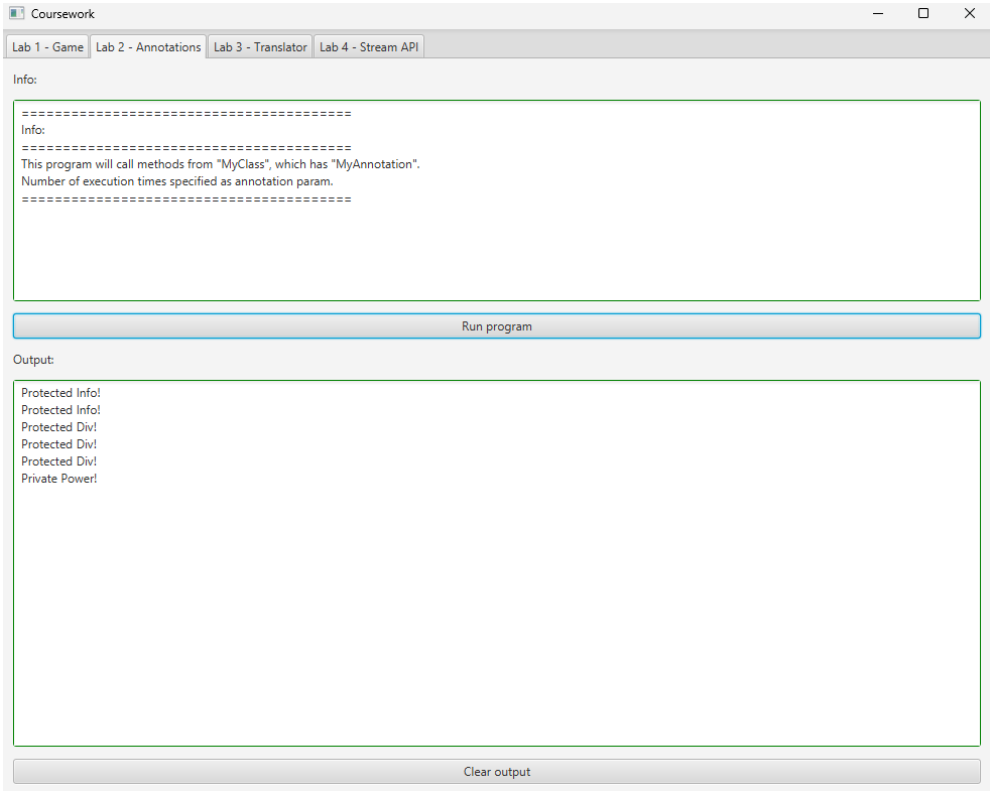


Рисунок 5. Результат выполнения лабораторной работы №2

Лабораторная работа 3 – Translator

Для корректного выполнения лабораторной работы №3 для начала, по заданию, следует выбрать файл словаря. Выбор реализован путем написания пути до файла словаря, при этом есть возможность выбрать сразу несколько словарей одновременно, в примере было выбрано два файла словаря.

Проверку работоспособности переводчика проведем на предложении “bird fly in sky fish in sea”. По итогу запуска работы видно, что программа перевела только те слова что присутствовали в словарях. Слова, которых там не было, остались на английском.

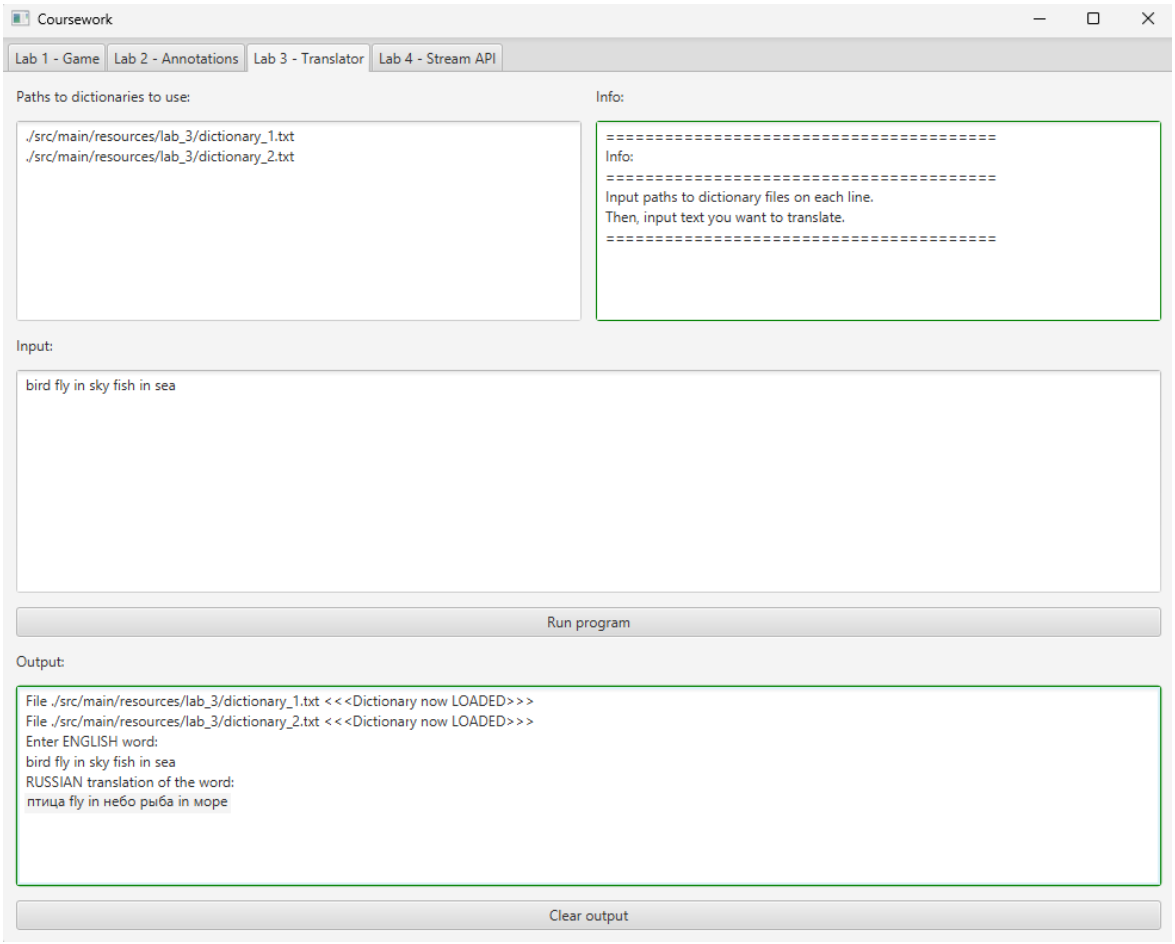


Рисунок 6. Результат выполнения лабораторной работы №3

Программа написана таким образом, что обрабатывает ошибки, если в поле выбора файла ввести несуществующий файл или директорию, либо попытаться выбрать не txt файл. Пример ошибок показан на рисунках 7-8.

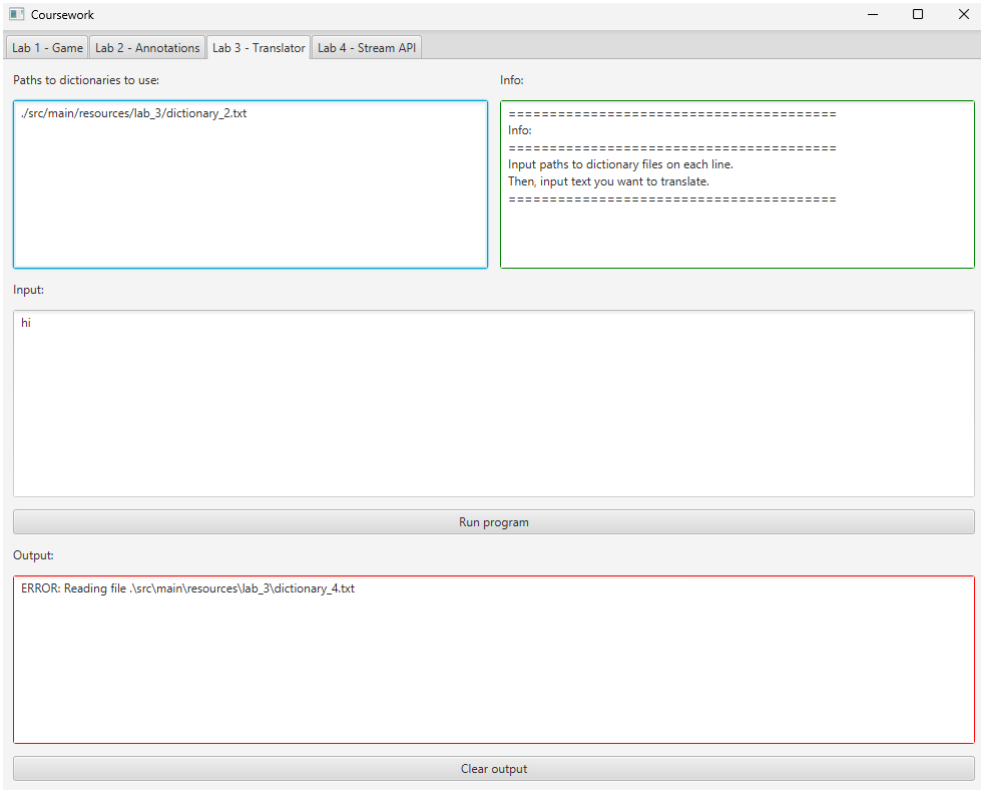


Рисунок 7. Ошибка при выборе не существующего файла.

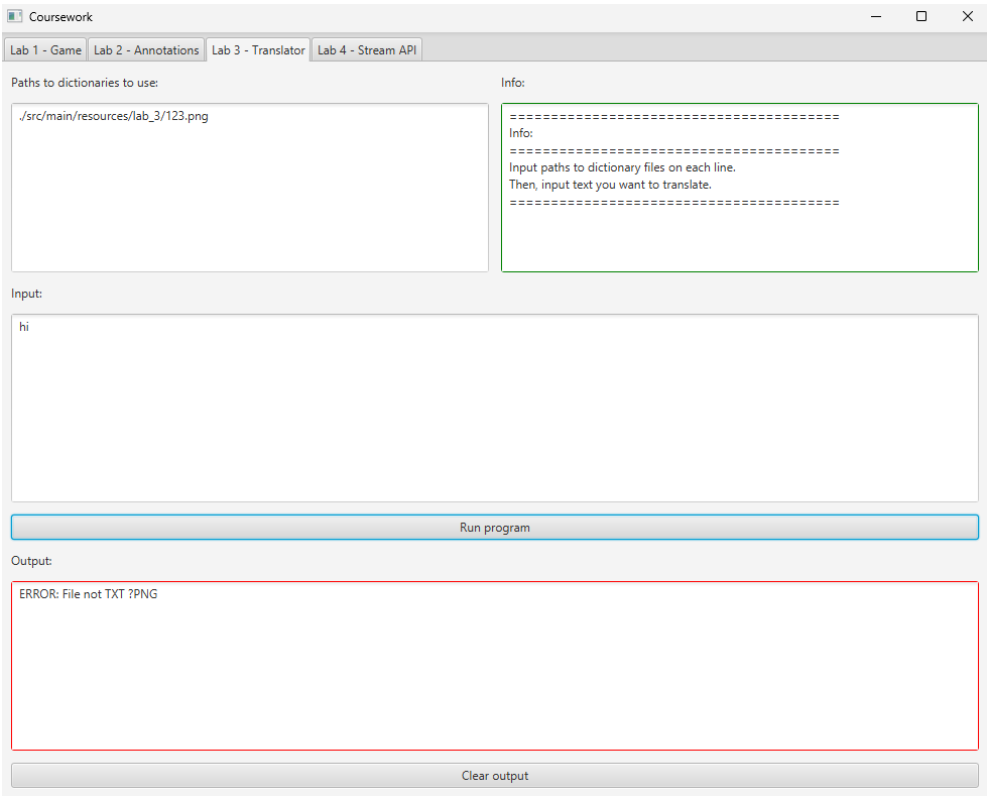


Рисунок 6. Ошибка при выборе не правильного расширения файла.

Лабораторная работа 4 – Stream API

В поле ввода входных данных на обработку введем подходящие и проверим работоспособность программы. Показано на рисунке 7.

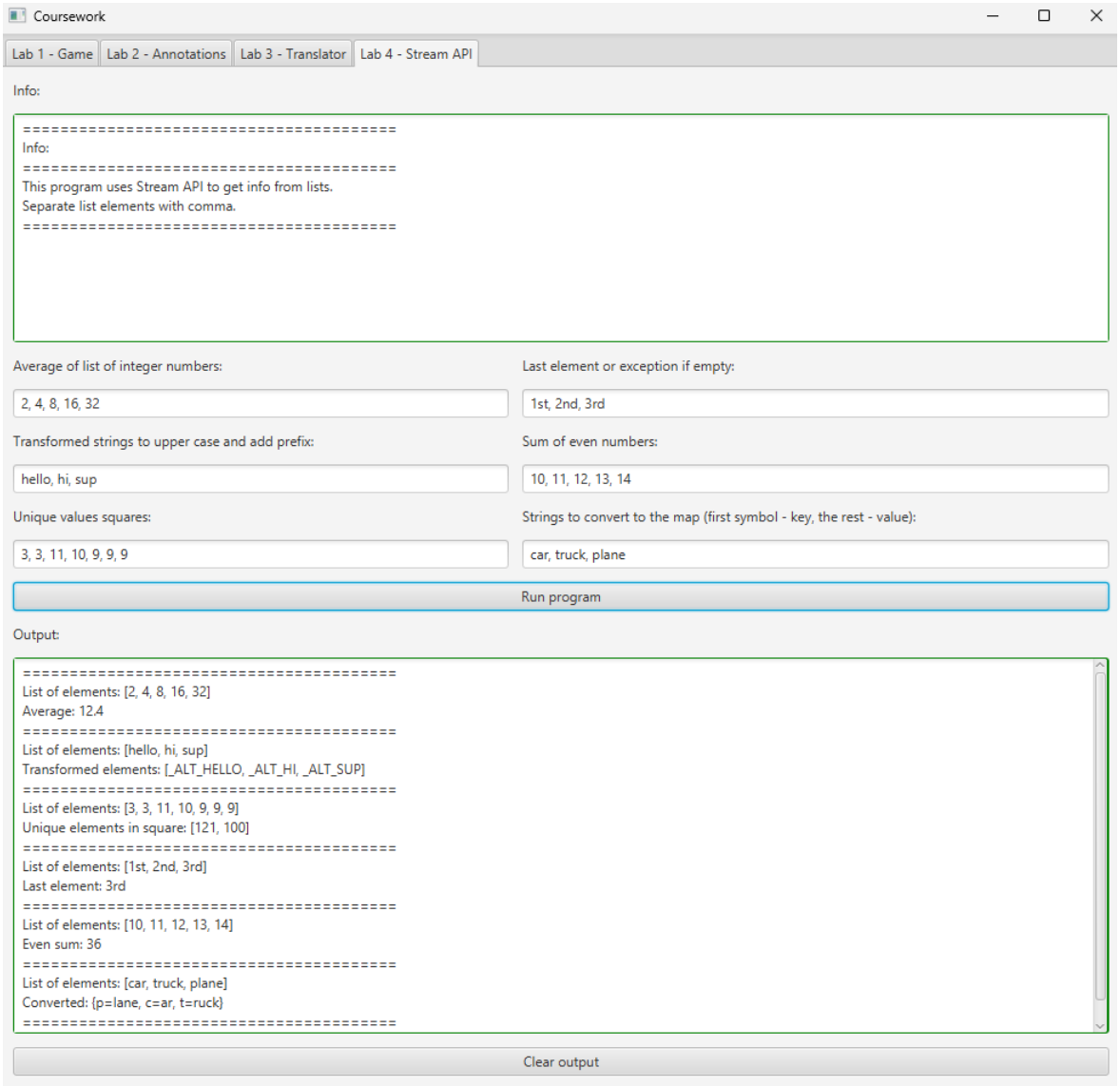


Рисунок 7. Результат выполнения лабораторной работы №4

Как видно на рисунке, код выполняет все свои функции, теперь попробуем специально ввести не правильные значения чтобы получить сообщения об ошибках ввода. Для этого в поля, где требуется численные значения, я ввел букву “x” и сразу получил ошибку, показанную на рисунке 8.

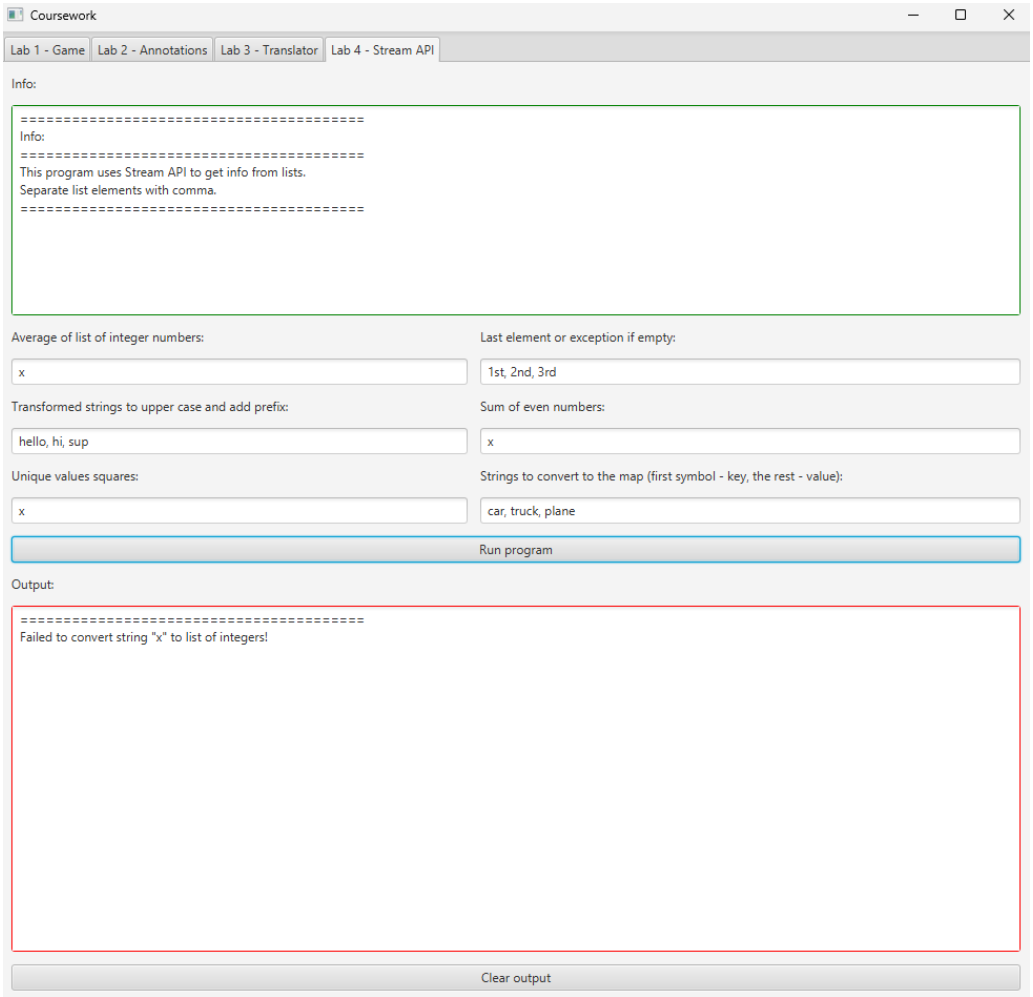


Рисунок 8. Ошибка об неправильном вводе.

Выводы

На основе выполненной работы, было разработано приложение с графическим интерфейсом для выполнения лабораторных работ 1-4 на языке программирования Java, при помощи графического расширения javafx и с использованием IntelliJ IDEA.

Целью данной разработки было предоставление пользователю удобного и понятного средства для выполнения лабораторных работ, включая выбор задания, ввод входных данных и получение результатов, а также обработку некорректного ввода.

Программа демонстрирует правильное исполнение всех лабораторных работ. Практические результаты совпадают с теоретическими ожиданиями.

Таким образом, все цели курсового проекта были достигнуты.

Листинг кода всего проекта

Основной код курсового проека:

Main:

```
package coursework;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;

import java.io.File;
import java.io.IOException;

public class Main extends Application {
    public static void main(String[] args) {
        launch(args);
    }

    @Override
    public void start(Stage stage) throws IOException {
        stage.setWidth(1000);
        stage.setHeight(800);

        String projectPath = "E:/torrent/Lab_kurs";
        File fxmlFile = new File(projectPath +
"/src/main/resources/coursework/MainWindow.fxml");

        if (!fxmlFile.exists()) {
            System.err.println("FXML файл не найден по пути: " +
fxmlFile.getAbsolutePath());

            File resourcesDir = new File(projectPath +
"/src/main/resources");
            if (resourcesDir.exists()) {
                System.out.println("Содержимое папки resources:");
                for (File file : resourcesDir.listFiles()) {
                    System.out.println(" - " + file.getName() +
(file.isDirectory() ? " [папка]" : ""));
                }
            }

            File courseworkDir = new File(projectPath +
"/src/main/resources/coursework");
            if (courseworkDir.exists()) {
                System.out.println("Содержимое папки coursework:");
                for (File file : courseworkDir.listFiles()) {
                    System.out.println(" - " + file.getName());
                }
            }

            throw new RuntimeException("FXML файл не найден! " +
fxmlFile.getAbsolutePath());
        }

        System.out.println("Успешно! FXML найден по пути: " +
fxmlFile.getAbsolutePath());
```

```
        FXMLLoader loader = new FXMLLoader(fxmlFile.toURI().toURL());
        Parent root = loader.load();

        Scene scene = new Scene(root, stage.getWidth(), stage.getHeight());
        stage.setScene(scene);
        stage.setTitle("Coursework");
        stage.show();
    }
}
```

MainWindowController:

```
package coursework;

import javafx.fxml.FXML;
import javafx.scene.control.Alert;
import javafx.scene.control.TextArea;
import javafx.scene.control.TextField;

import java.io.*;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

public class MainWindowController {
    @FXML
    private TextArea lab1Info, lab2Info, lab3Info, lab4Info, lab1Output,
lab2Output, lab3Output, lab4Output, lab3Input, lab3DictionariesList;
    @FXML
    private TextField lab1Input, lab4InputAverageOfList,
lab4InputTransformedStrings, lab4InputUniqueSquares, lab4InputLastElement,
lab4InputSumOfEvenNumbers, lab4InputConvertedMap;

    @FXML
    public void initialize() {
        // Set lab information by calling it with "--info" argument
        runLab("lab_1.Main", new String[]{"--info"}, lab1Info);
        runLab("lab_2.Main", new String[]{"--info"}, lab2Info);
        runLab("lab_3.Main", new String[]{"--info"}, lab3Info);
        runLab("lab_4.Main", new String[]{"--info"}, lab4Info);
    }

    public void startLab1() {
        runLab("lab_1.Main", new String[]{lab1Input.getText()}, lab1Output);
    }

    public void startLab2() {
        runLab("lab_2.Main", new String[]{}, lab2Output);
    }

    public void startLab3() {
        ArrayList<String> options = new ArrayList<>();
        options.add(lab3Input.getText());


options.addAll(Arrays.stream(lab3DictionariesList.getText().split("\n")).filt
er(s -> !s.trim().isEmpty()).toList());

        runLab("lab_3.Main", options.toArray(new String[0]), lab3Output);
    }
```

```
    public void startLab4() {
        runLab("lab_4.Main", new String[]{
                // We pass all values as string and then will convert them to
arrays inside lab 4
                lab4InputAverageOfList.getText(),
                lab4InputTransformedStrings.getText(),
                lab4InputUniqueSquares.getText(),
                lab4InputLastElement.getText(),
                lab4InputSumOfEvenNumbers.getText(),
                lab4InputConvertedMap.getText(),
        }, lab4Output);
    }

    public void clearOutputForLab1() {
        lab1Output.clear();
    }

    public void clearOutputForLab2() {
        lab2Output.clear();
    }

    public void clearOutputForLab3() {
        lab3Output.clear();
    }

    public void clearOutputForLab4() {
        lab4Output.clear();
    }

    private void runLab(String className, String[] args, TextArea outputArea)
{

        List<String> command = new ArrayList<>();


        String javaHome = System.getProperty("java.home");
        String javaExecutable = javaHome + File.separator + "bin" +
File.separator + "java";
        command.add(javaExecutable);

        command.add("-classpath");

        command.add("./target/classes");

        command.add(className);
        Collections.addAll(command, args);


        ProcessBuilder processBuilder = new ProcessBuilder(command);
        Process process;
        try {
            process = processBuilder.start();
        } catch (IOException e) {
            showError("Error while starting lab process: " + e.getMessage());
            return;
        }

        StringBuilder outputTextBuilder = new StringBuilder();

        BufferedReader outReader = new BufferedReader(new
InputStreamReader(process.getInputStream()));
        BufferedReader errReader = new BufferedReader(new
InputStreamReader(process.getErrorStream()));
```

```
        try {
            String outLine;
            String errLine;
            do {

                outLine = outReader.readLine();
                if (outLine != null) {
                    outputTextBuilder.append(outLine).append("\n");
                }

                errLine = errReader.readLine();
                if (errLine != null) {
                    outputTextBuilder.append(errLine).append("\n");
                }
            } while (outLine != null || errLine != null);
        } catch (IOException e) {
            showError("Error while reading lab process standard output: " +
e.getMessage());
            return;
        }

        int exitCode;
        try {
            exitCode = process.waitFor();
        } catch (InterruptedException e) {
            showError("Error while waiting for lab process to exit: " +
e.getMessage());
            return;
        }

        if (exitCode == 0) {
            outputArea.setStyle("-fx-background-color: green;");
        } else {
            outputArea.setStyle("-fx-background-color: red;");
        }

        outputArea.setText(outputTextBuilder.toString());
    }

    private void showError(String errorMessage) {
        Alert alert = new Alert(Alert.AlertType.ERROR);
        alert.setTitle("Error");
        alert.setHeaderText(null);
        alert.setContentText(errorMessage);
        alert.showAndWait();
    }
}
```

Lab_1:

Main:

```
package lab_1;

import lab_1.moving_strategies.*;

import java.util.Arrays;

public class Main {
    private static final Hero tamerlan = new Hero("Tamerlan", new
MovingStrategyWalking());
```

```
    private static final Hero anton = new Hero("Anton", new
MovingStrategyRunning());
    private static final Hero hero = new Hero("Hero", new
MovingStrategyNone());

    public static void printInfo () {
        System.out.println("========================================");
        System.out.println("Start positions:");
        System.out.println("========================================");
        System.out.println(tamerlan);
        System.out.println(anton);
        System.out.println(hero);

        System.out.println("========================================");
        System.out.println("Actions:");
        System.out.println("========================================");
        System.out.println("0 - exit");
        System.out.println("1 - do not move");
        System.out.println("2 - walk");
        System.out.println("3 - run");
        System.out.println("4 - move by horse");
        System.out.println("5 - move by starship");
        System.out.println("========================================");
    }

    public static void main(String[] args) {
        if (Arrays.stream(args).toList().contains("--info")) {
            printInfo();
            System.exit(0);
        }

        if (args.length == 0) {
            System.out.println("Please enter your actions as string as first
argument to the program.");
            System.exit(1);
        }

        String moves = args[0];

        if (moves.isEmpty()) {
            System.out.println("Please enter your actions as string as first
argument to the program.");
            System.exit(1);
        }

        System.out.println("========================================");

        for_cycle:
        for (int i = 0; i < moves.length(); i++) {
            System.out.print("Your turn. Choose action: ");
            char command = moves.charAt(i);
            System.out.println(command);

            switch (command) {
                case '0':
                    break for_cycle;
                case '1':
                    hero.setMovingStrategy(new MovingStrategyNone());
                    break;
                case '2':
                    hero.setMovingStrategy(new MovingStrategyWalking());
                    break;
                case '3':
                    hero.setMovingStrategy(new MovingStrategyRunning());
```

```
                    break;
                case '4':
                    hero.setMovingStrategy(new MovingStrategyOnHorse());
                    break;
                case '5':
                    hero.setMovingStrategy(new MovingStrategySpaceFlying());
                    break;
                default:
                    System.out.println("Unknown action: " + command + ".");
                    System.exit(1);
            }

            System.out.println("========================================");
            System.out.println("Everyone moved:");

            tamerlan.move();
            anton.move();
            hero.move();

            System.out.println(tamerlan);
            System.out.println(anton);
            System.out.println(hero);
        }

        System.out.println("========================================");
    }
}
```

Hero:

```
package lab_1;

import lab_1.moving_strategies.IMovingStrategy;

import java.awt.*;

public class Hero {
    private final String name;

    private final Point location;

    private IMovingStrategy movingStrategy;

    public Hero(String name, IMovingStrategy movingStrategy) {
        this.name = name;
        this.location = new Point(0, 0);
        this.movingStrategy = movingStrategy;
    }

    @Override
    public String toString() {
        return "Hero name: " + name + ". Location: " + location.getX() + ", "
+ location.getY() + ". Moving strategy: " +
movingStrategy.getClass().getSimpleName();
    }

    public IMovingStrategy getMovingStrategy() {
        return movingStrategy;
    }

    public void setMovingStrategy(IMovingStrategy movingStrategy) {
        this.movingStrategy = movingStrategy;
```

```
    }

    void move() {
        this.movingStrategy.move(this.location);
    }
}
```

Moving Strategies:

MovingStrategyNone:

```
                        package lab_1.moving_strategies;

import java.awt.*;

public class MovingStrategyNone implements IMovingStrategy {
    @Override
    public void move(Point location) {
        // Nothing
    }
}
```

MovingStrategyOnHorse:

```
package lab_1.moving_strategies;

import java.awt.*;

public class MovingStrategyOnHorse implements IMovingStrategy {
    @Override
    public void move(Point location) {
        location.setLocation(location.getX() + 30, location.getY());
    }
}
```

MovingStrategyRunning:

```
package lab_1.moving_strategies;

import java.awt.*;

public class MovingStrategyRunning implements IMovingStrategy {
    @Override
    public void move(Point location) {
        location.setLocation(location.getX() + 10, location.getY());
    }
}
```

MovingStrategySpaceFlying:

```
package lab_1.moving_strategies;

import java.awt.*;

public class MovingStrategySpaceFlying implements IMovingStrategy {
    @Override
    public void move(Point location) {
```

```
        location.setLocation(location.getX() + 1000, location.getY());
    }
}
```

MovingStrategyWalking:

```
package lab_1.moving_strategies;

import java.awt.*;

public class MovingStrategyWalking implements IMovingStrategy {
    @Override
    public void move(Point location) {
        location.setLocation(location.getX() + 5, location.getY());
    }
}
```

Lab_2:

Main:

```
package lab_2;

import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.lang.reflect.Modifier;
import java.util.Arrays;

public class Main {
    public static void printInfo () {
        System.out.println("========================================");
        System.out.println("Info:");
        System.out.println("========================================");
        System.out.println("This program will call methods from \"MyClass\",
which has \"MyAnnotation\".");
        System.out.println("Number of execution times specified as annotation
param.");
        System.out.println("========================================");
    }

    public static void main(String[] args) {
        if (Arrays.stream(args).toList().contains("--info")) {
            printInfo();
            System.exit(0);
        }

        MyClass obj = new MyClass();

        Method[] methods = MyClass.class.getDeclaredMethods();

        for (Method method : methods) {
            if (!method.isAnnotationPresent(MyAnnotation.class)) {
                continue;
            }

            if (!Modifier.isProtected(method.getModifiers()) &&
!Modifier.isPrivate(method.getModifiers())) {
                continue;
            }
```

```
            method.setAccessible(true);

            int timesToExecute =
method.getAnnotation(MyAnnotation.class).timesToExecute();

            for (int executionId = 0; executionId < timesToExecute;
executionId++) {
                Class<?>[] parameterTypes = method.getParameterTypes();

                Object[] parameters = new Object[parameterTypes.length];
                for (int parameterId = 0; parameterId <
parameterTypes.length; parameterId++) {
                    Class<?> parameterType = parameterTypes[parameterId];
                    if (parameterType == int.class) {
                        parameters[parameterId] = 5;
                    } else if (parameterType == double.class) {
                        parameters[parameterId] = 7.4;
                    } else if (parameterType == float.class) {
                        parameters[parameterId] = 8.3f;
                    } else if (parameterType == boolean.class) {
                        parameters[parameterId] = true;
                    } else if (parameterType == String.class) {
                        parameters[parameterId] = "Lalala";
                    } else {
                        throw new RuntimeException("Parameter type " +
parameterType.getName() + " not supported!");
                    }
                }

                try {
                    method.invoke(obj, parameters);
                } catch (IllegalAccessException | InvocationTargetException
e) {
                    throw new RuntimeException(e);
                }
            }
        }
    }
}
```

My Annotation:

```
package lab_2;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface MyAnnotation {
    int timesToExecute() default 0;
}
```

MyClass:

```
package lab_2;

public class MyClass {
```

```
    public int sum(int a, int b) {
        System.out.println("Public Sum!");
        return a + b;
    }

    @MyAnnotation(timesToExecute = 2)
    public double sqrt(double a) {
        System.out.println("Public Sqrt!");
        return Math.sqrt(a);
    }

    public void info() {
        System.out.println("Hello, World!");
    }

    protected int sub(int a, int b) {
        System.out.println("Protected Sub!");
        return a - b;
    }

    @MyAnnotation(timesToExecute = 3)
    protected double div(double a, double b) {
        System.out.println("Protected Div!");
        return a / b;
    }

    @MyAnnotation(timesToExecute = 2)
    protected void superInfo() {
        System.out.println("Protected Info!");
    }

    @MyAnnotation(timesToExecute = 0)
    private int mul(int a, int b) {
        System.out.println("Private Mul!");
        return a * b;
    }

    @MyAnnotation(timesToExecute = 1)
    private double power(double a, double b) {
        System.out.println("Private Power!");
        return Math.pow(a, b);
    }

    private void secretInfo() {
        System.out.println("Private Info!");
    }
}
```

Lab_3:

Main:

```
package lab_3;

import lab_3.exceptions.FileReadException;
import lab_3.exceptions.InvalidFileFormatException;

import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void printInfo () {
```

```
        System.out.println("========================================");
        System.out.println("Info:");
        System.out.println("========================================");
        System.out.println("Input paths to dictionary files on each line.");
        System.out.println("Then, input text you want to translate.");
        System.out.println("========================================");
    }

    public static void main(String[] args) {
        if (Arrays.stream(args).toList().contains("--info")) {
            printInfo();
            System.exit(0);
        }

        if (args.length == 0) {
            System.out.println("Please enter your text for translation as
first argument to the program.");
            System.exit(1);
        } else if (args.length == 1) {
            System.out.println("Please specify dictionaries paths as
arguments to the program after your text.");
            System.exit(1);
        }

        Dictionary translator = new Dictionary();
        for (int dictionary_id = 1; dictionary_id < args.length;
dictionary_id++) {
            String dictionaryFilePath = args[dictionary_id];
            try {
                translator.loadTranslationsFromFile(dictionaryFilePath);
            } catch (InvalidFileFormatException | FileReadException e) {
                System.out.println(e.getMessage());
                System.exit(1);
            }
            System.out.println("File " + dictionaryFilePath + " <<<Dictionary
now LOADED>>>");
        }

        System.out.println("Enter ENGLISH word:");

        String input = args[0];
        System.out.println(input);

        System.out.println("RUSSIAN translation of the word:");
        System.out.println(translator.translateText(input));
    }

}
```

Dictionary:

```
package lab_3;

import lab_3.exceptions.FileReadException;
import lab_3.exceptions.InvalidFileFormatException;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.nio.file.Paths;
import java.util.Comparator;
import java.util.HashMap;
```

```java
import java.util.Map;

public class Dictionary {

    private final Map<String, String> dictionary = new HashMap<>();

    public void loadTranslationsFromFile(String filePath) throws
InvalidFileFormatException, FileReadException {

        try (BufferedReader reader = new BufferedReader(new
FileReader(Paths.get(filePath).toFile()))) {
            String line;
            while ((line = reader.readLine()) != null) {

                String[] parts = line.split("\\|");
                if (parts.length != 2) {
                    throw new InvalidFileFormatException("ERROR: File not TXT
" + line);
                }

                dictionary.put(parts[0].trim().toLowerCase(),
parts[1].trim().toLowerCase());
            }
        } catch (IOException e) {
            throw new FileReadException("ERROR: Reading file " +
e.getMessage());
        }
    }

    public String translateText(String input) {
        String[] words = input.split("\\s+");

        StringBuilder result = new StringBuilder();

        for (int wordId = 0; wordId < words.length; wordId++) {
            String word = words[wordId].toLowerCase();

            String bestMatch = findBestMatch(word);
            if (bestMatch != null) {

                result.append(dictionary.get(bestMatch));
            } else {

                result.append(words[wordId]);
            }

            if (wordId < words.length - 1) {
                result.append(" ");
            }
        }

        return result.toString();
    }

    private String findBestMatch(String input) {

        return dictionary.keySet().stream()
                .filter(input::startsWith)
                .max(Comparator.comparingInt(String::length))
                .orElse(null);
    }
}
```

Exceptions:

FileReadException:

```
package lab_3.exceptions;

public class FileReadException extends Exception {
    public FileReadException(String message) {
        super(message);
    }
}
```

InvalidFileFormatException:

```
        package lab_3.exceptions;

public class InvalidFileFormatException extends Exception {
    public InvalidFileFormatException(String message) {
        super(message);
    }
}
```

Lab_4:

Main:

```
package lab_4;

import java.util.*;
import java.util.stream.Collectors;

public class Main {
    public static void printInfo () {
        System.out.println("========================================");
        System.out.println("Info:");
        System.out.println("========================================");
        System.out.println("This program uses Stream API to get info from
lists.");
        System.out.println("Separate list elements with comma.");
        System.out.println("========================================");
    }

    public static void main(String[] args) {
        if (Arrays.stream(args).toList().contains("--info")) {
            printInfo();
            System.exit(0);
        }

        if (args.length < 6) {
            System.out.println("You must specify all the arrays");
            System.exit(1);
        }


        System.out.println("========================================");
        List<Integer> numbers;
        try {
            numbers =
Arrays.stream(args[0].split("\\s*,\\s*")).map(Integer::parseInt).toList();
        } catch (NumberFormatException e) {
            System.out.println("Failed to convert string \"" + args[0] + "\"
to list of integers!");
```

```
            System.exit(1);
            return;
        }
        System.out.println("List of elements: " + numbers);
        System.out.println("Average: " + averageOfList(numbers).orElse(0.0));

        System.out.println("========================================");
        List<String> strings =
Arrays.stream(args[1].split("\\s*,\\s*")).toList();
        System.out.println("List of elements: " + strings);
        System.out.println("Transformed elements: " +
transformStrings(strings));

        System.out.println("========================================");
        List<Integer> duplicateNumbers;
        try {
            duplicateNumbers =
Arrays.stream(args[2].split("\\s*,\\s*")).map(Integer::parseInt).toList();
        } catch (NumberFormatException e) {
            System.out.println("Failed to convert string \"" + args[2] + "\"
to list of integers!");
            System.exit(1);
            return;
        }
        System.out.println("List of elements: " + duplicateNumbers);
        System.out.println("Unique elements in square: " +
uniqueSquares(duplicateNumbers));

        System.out.println("========================================");
        List<String> moreStrings =
Arrays.stream(args[3].split("\\s*,\\s*")).toList();
        System.out.println("List of elements: " + moreStrings);
        System.out.println("Last element: " + getLastElement(moreStrings));

        System.out.println("========================================");
        int[] numArray;
        try {
            numArray =
Arrays.stream(args[4].split("\\s*,\\s*")).mapToInt(Integer::parseInt).toArray
();
        } catch (NumberFormatException e) {
            System.out.println("Failed to convert string \"" + args[4] + "\"
to array of integers!");
            System.exit(1);
            return;
        }
        System.out.println("List of elements: " + Arrays.toString(numArray));
        System.out.println("Even sum: " + sumOfEvenNumbers(numArray));

        System.out.println("========================================");
        List<String> stringList =
Arrays.stream(args[5].split("\\s*,\\s*")).toList();
        System.out.println("List of elements: " + stringList);
        System.out.println("Converted: " + convertToMap(stringList));

        System.out.println("========================================");
    }

    public static OptionalDouble averageOfList(List<Integer> numbers) {
        return numbers.stream()
                .mapToInt(Integer::intValue)
                .average();
    }
```

```
    public static List<String> transformStrings(List<String> strings) {
        return strings.stream()
                .map(s -> "_ALT_" + s.toUpperCase())
                .collect(Collectors.toList());
    }

    public static List<Integer> uniqueSquares(List<Integer> numbers) {
        return numbers.stream()
                .filter(n -> Collections.frequency(numbers, n) == 1)
                .map(n -> n * n)
                .collect(Collectors.toList());
    }

    public static <T> T getLastElement(Collection<T> collection) {
        return collection.stream()
                .reduce((first, second) -> second)
                .orElseThrow(() -> new NoSuchElementException("Collection is
empty"));
    }

    public static int sumOfEvenNumbers(int[] numbers) {
        return Arrays.stream(numbers)
                .filter(n -> n % 2 == 0)
                .sum();
    }

    public static Map<Character, String> convertToMap(List<String> strings) {
        return strings.stream()
                .filter(s -> s.length() > 0)
                .collect(Collectors.toMap(s -> s.charAt(0), s ->
s.substring(1)));
    }
}
```